Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2 Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3 Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

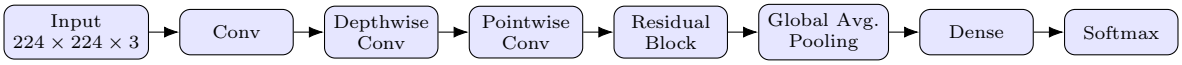
4 MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5 Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

6 Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

7 Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

8 Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

9 CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size

- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

10 Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

$$\text{Pretrained MobileNetV2} \rightarrow \text{Freeze Base} \rightarrow \text{New Classifier}$$

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

$$\text{Pretrained MobileNetV2} \rightarrow \text{Unfreeze Selected Layers} \rightarrow \text{Small Learning Rate}$$

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

11 K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

12 Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

13 Source Code

Source: Github
Github Repository Link:
`https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab5_MobileNetV2/DL_Lab5.ipynb`

14 Plots and Inference

1. **Sample Oxford-IIIT Pet Dataset images**

Figure 1: Sample Images

Inference:
The figures show sample images from the Oxford IIIT Pet Dataset. The dataset contains 7390 images. The images belong to 37 different breeds of cats and dogs. Each image is 224 x 224 pixels and contains three color channels (RGB)

2. **Weight Initialization: Training Loss vs Epoch**

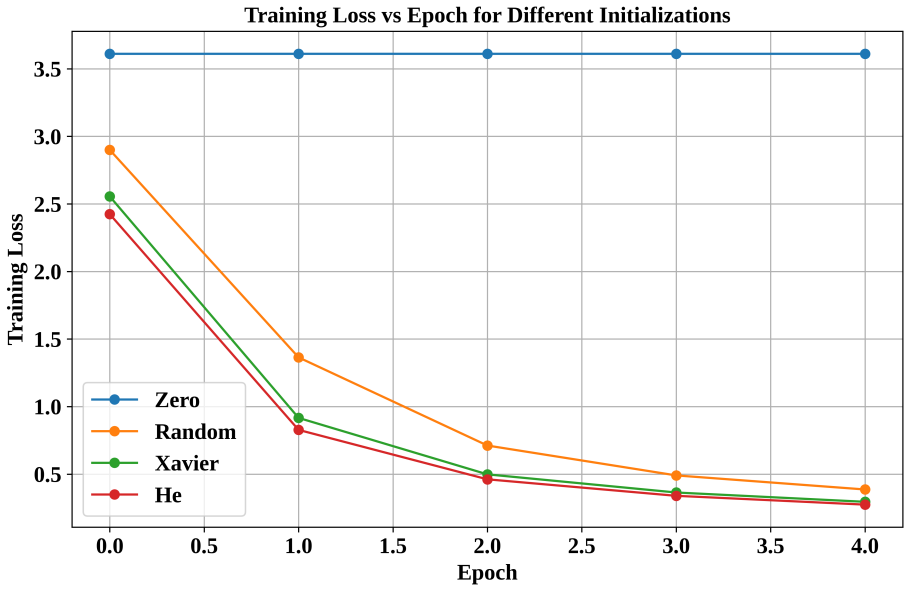


Figure 2: Training Loss vs Epoch for Different Initializations

Inference:
The plot shows the training loss over 5 epochs for different weight initialization methods such as Zero, Random, Xavier, and He initialization. Random, Xavier, and He initialization show a steady decrease in training loss, while Zero initialization remains almost constant showing high loss of value approx. 3.6 . Proper weight initialization breaks symmetry and allows effective gradient-based learning, whereas zero initialization causes neurons to learn identical features and prevents effective training.

3. **Weight Initialization: Validation Accuracy vs Epoch**

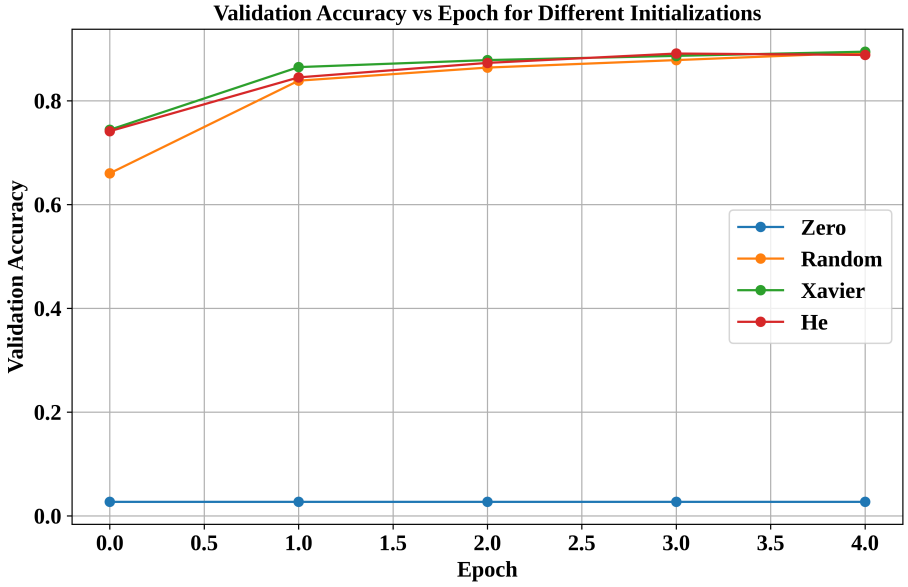


Figure 3: Validation Accuracy vs Epoch for Different Initializations

Inference:
The plot shows the validation accuracy over 5 epochs for different weight initialization such as Zero, Random, Xavier, He. Xavier, He, and Random initializations rapidly increase in validation accuracy converging near 89 percent by epoch 5. Zero initialization fails to improve, remaining at validation accuracy of value approx. 0 percent across all epochs. Zero initialization causes weight symmetry, forcing all neurons to receive identical gradient

updates and preventing feature learning. On the other hand, Xavier and He initialization break symmetry and prevent vanishing or exploding gradients, allowing effective optimization from epoch 0.

4. **No Regularization**

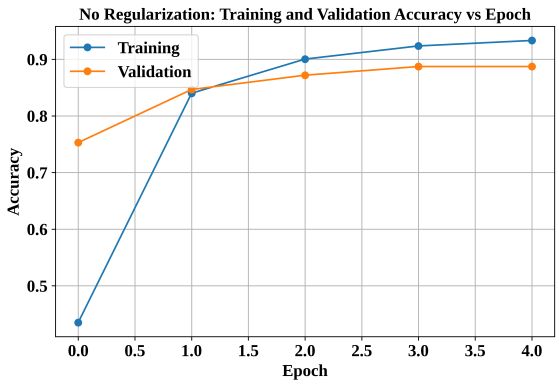


Figure 4: Training and Validation Accuracy vs Epoch

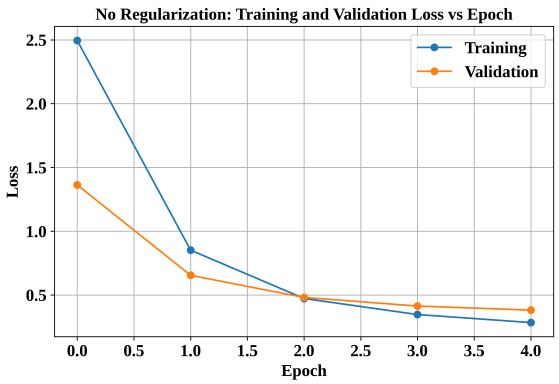


Figure 5: Training and Validation Loss vs Epoch

Training and Validation Accuracy vs Epoch Inference:
The plot compares training accuracy and validation accuracy across 5 epochs for models trained with no regularization. The validation accuracy shows stable increase whereas training accuracy increases sharply then stabilises.

Training and Validation Loss vs Epoch Inference:
The plot compares training and validation loss across 5 epochs for models trained with no regularization. Both the curves shows steady decrease and reaches minimum of approx. 0.5 loss value.

5. **L2 Regularization**

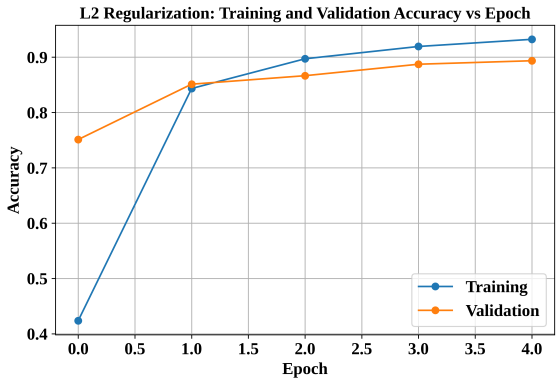


Figure 6: Training and Validation Accuracy vs Epoch

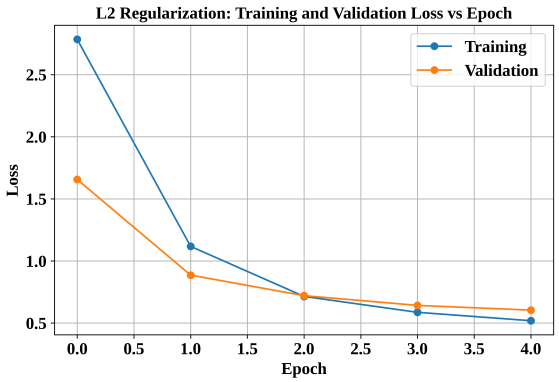


Figure 7: Training and Validation Loss vs Epoch

Training and Validation Accuracy vs Epoch Inference:
The plot compares training accuracy and validation accuracy across 5 epochs for models trained with L2 regularization. The validation accuracy shows stable increase whereas training accuracy increases sharply then stabilises.

Training and Validation Loss vs Epoch Inference:
The plot compares training and validation loss across 5 epochs for models trained with L2 regularization. Both the curves shows steady decrease and reaches minimum of approx. 0.5 loss value.

6. Dropout

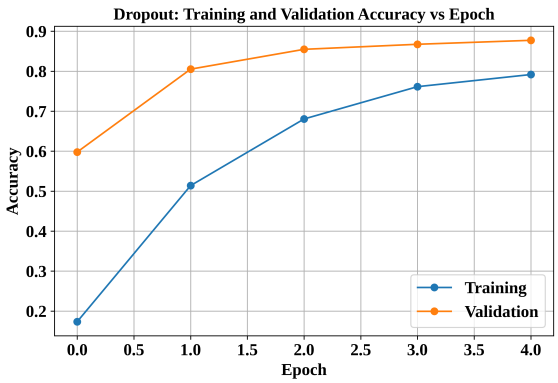


Figure 8: Training and Validation Accuracy vs Epoch

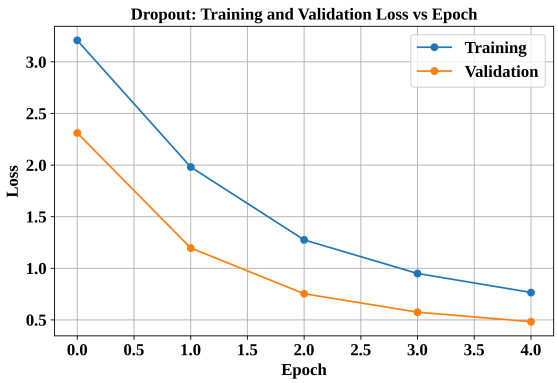


Figure 9: Training and Validation Loss vs Epoch

Training and Validation Accuracy vs Epoch Inference:
The plot compares training and validation accuracy across 5 epochs for models trained with dropout. Both the curves shows steady increase with validation curve having higher accuracy consistently.

Training and Validation Loss vs Epoch Inference:
The plot compares training and validation loss across 5 epochs for models trained with dropout. Both the curves shows steady decrease with validation curve having lower loss consistently.

7. Batch Normalization

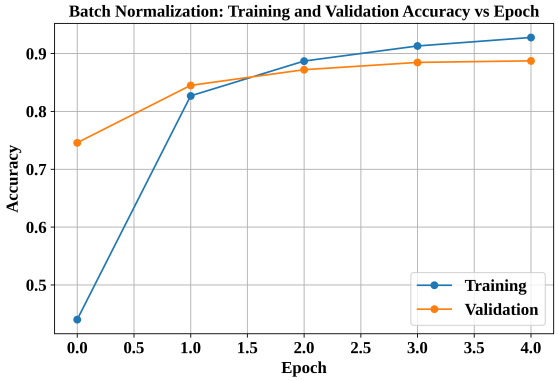


Figure 10: Training and Validation Accuracy vs Epoch

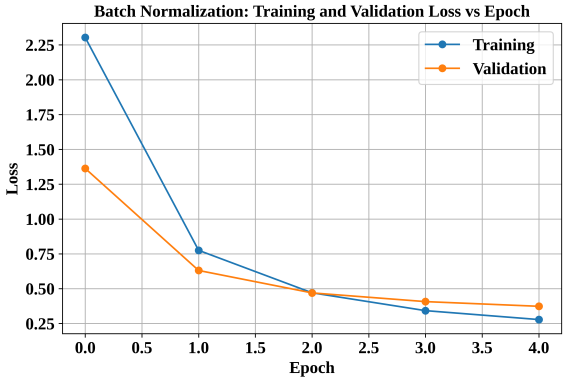


Figure 11: Training and Validation Loss vs Epoch

Training and Validation Accuracy vs Epoch Inference:
The plot compares training and validation accuracy across 5 epochs for models trained with batch normalization. Both the curves shows steady increase with validation curve having higher accuracy consistently.

Training and Validation Loss vs Epoch Inference:
The plot compares training and validation loss across 5 epochs for models trained with batch normalization. Both the curves shows steady decrease with training curve having lower loss by the end of 5th epoch.

8. **With vs Without Batch Normalization**

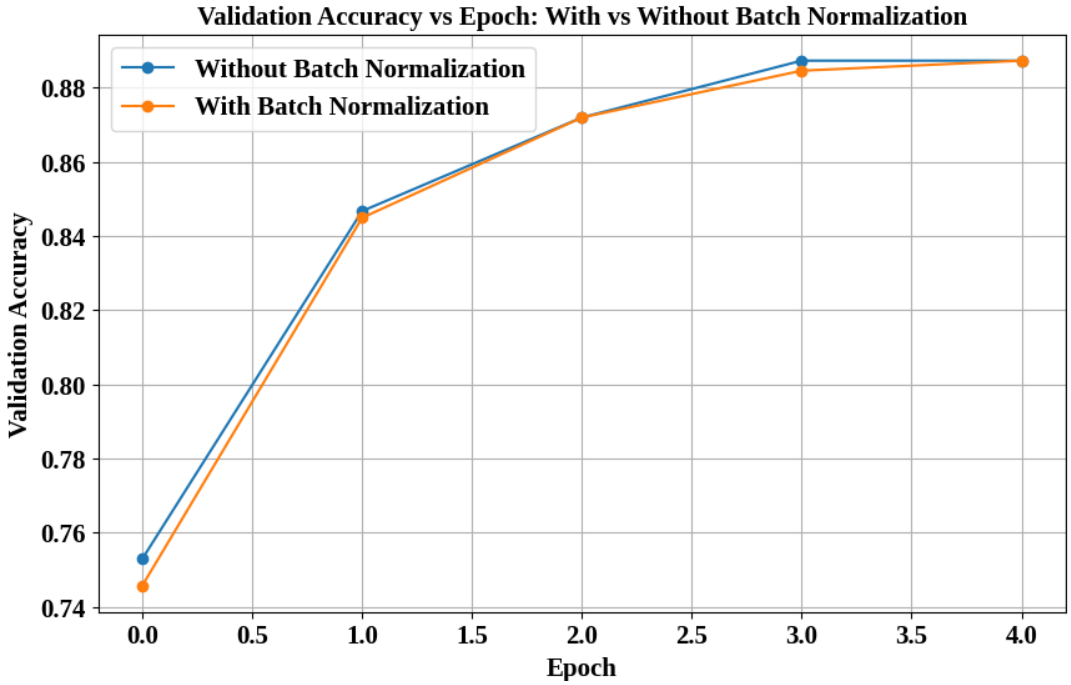


Figure 12: With vs Without Batch Normalization

Inference:
The plot shows the validation accuracy of a model with batch normalization and without batch normalization across 5 epochs. Both configurations follows almost identical trajectories reaching approx. 89 percent validation accuracy by the end of 5 epochs. Batch Normalization primarily stabilizes deeper networks or higher learning rates. For simpler architectures with smaller learning rates standard optimization already stabilizes quickly without extra normalization layers.

9. **Training Loss vs. Epoch for Different Optimizers**

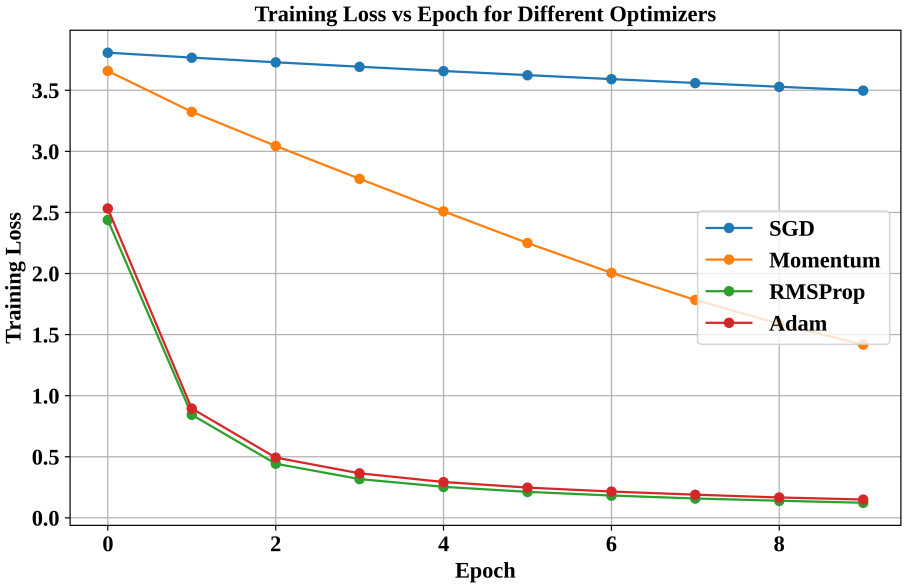


Figure 13: Training Loss vs. Epoch for Different Optimizers

Inference:
The plot compares the training loss of four optimization algorithms such as SGD, Momentum, RMSProp, Adam across 10 epochs. RMSProp and Adam experience a sharp drop in training loss within the first two epochs and reach a steady training loss. Momentum displays a steady linear decrease in training loss reaching a minimum of 1.5. SGD shows highest training loss of 3.5 byt the end of 10 epochs. Adaptive optimizers such as Adam and RMSProp dynamically update learning rates for each parameter. Hence they show rapid decrease in training loss. Momentum uses accumulated velocity to maintain steady updates, whereas SGD is shows poor performance due to inefficient step sizes.

10. **Validation Accuracy vs. Epoch for Different Optimizers**

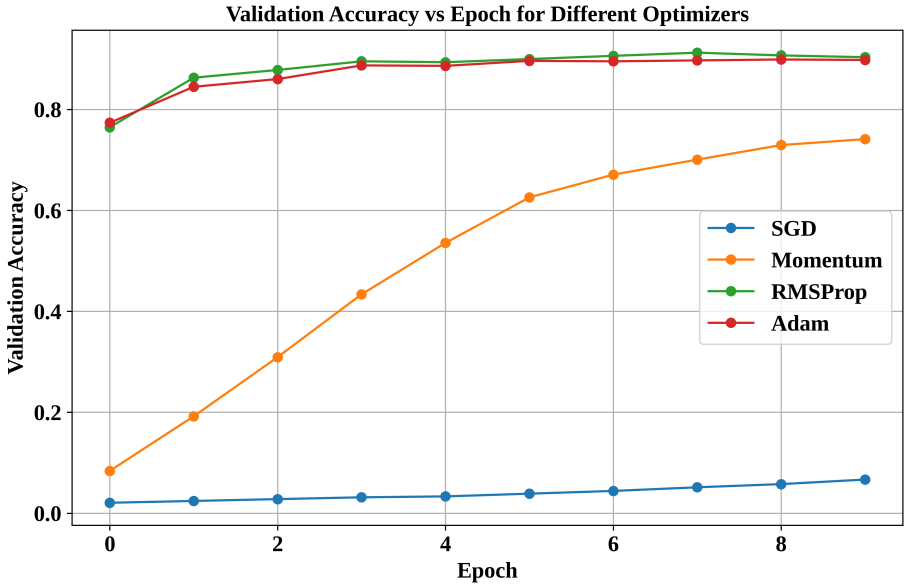


Figure 14: Validation Accuracy vs. Epoch for Different Optimizers

Inference:
The plot compares the validation accuracy of four optimization algorithms such as SGD, Momentum, RMSProp, Adam across 10 epochs. Adaptive optimizers such as Adam and RMSProp achieve rapid early convergence with high accuracy of approx. 85 percent. Momentum exhibits steady, moderate growth while SGD performs poorly with under 10 percent accuracy. This trend is observed because Adam and RMSProp dynamically adapt learning rates for individual parameters to quickly navigate steep gradients. Momentum uses velocity to accelerate past gradients, whereas SGD uses a fixed learning rate that struggles to make progress.

11. **Dropout Rate vs Validation Accuracy**

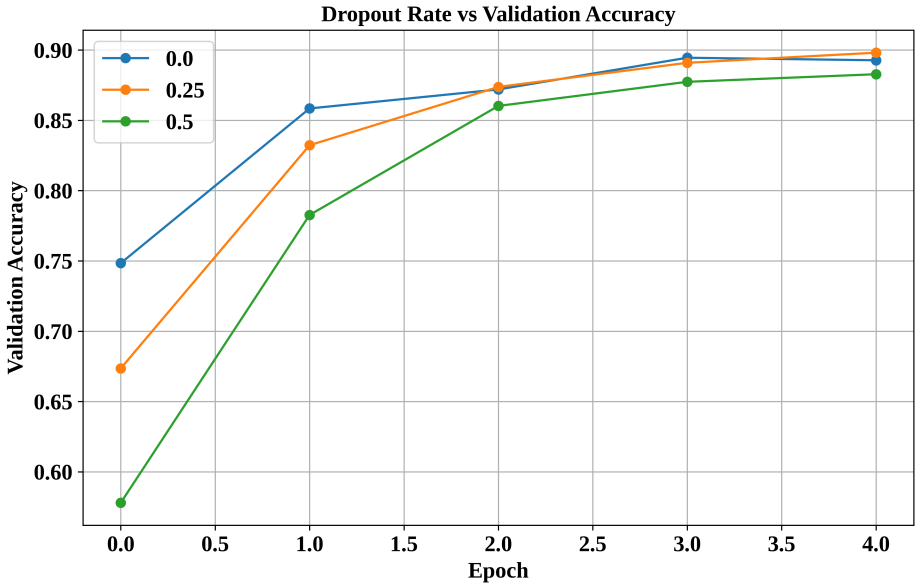


Figure 15: Dropout Rate vs Validation Accuracy

Inference:
The plot shows the comparison of validation accuray for different dropout rates (dropout rate = 0.0, 0.25, 0.5) over 5 epochs. Initially, no dropout shows highest validation accuracy. By the end of 5th epoch, model with dropout rate = 0.25 reaches the highest validation accuracy. The model with dropout rate = 0.5 has the lowest validation accuracy. By preventing neurons from co-adapting and relying on specific features, dropout rate = 0.25 reduces overfitting and allows the network to generalize better.

12. **Feature Extraction vs. Fine-Tuning**

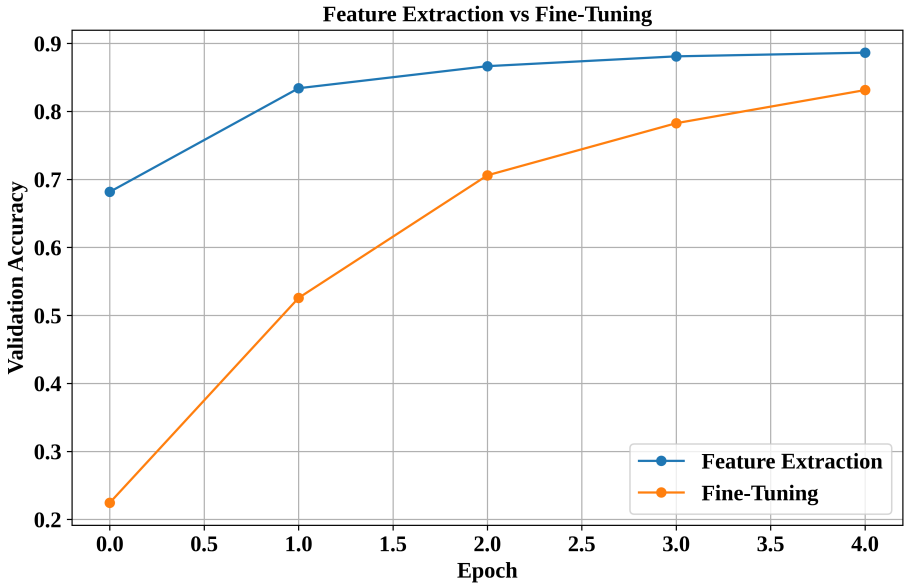


Figure 16: Feature Extraction vs. Fine-Tuning

Inference:
The plot shows the comparison of feature extraction and fine tuning. Feature Extraction starts at a much higher initial validation accuracy and maintains a clear accuracy advantage across all 5 epochs, reaching approx. 89 percent compared to Fine-Tuning's 80 percent. Feature Extraction freezes the feature extractor layers, preserving the strong general visual representations while only updating the final classification head. This prevents early instability. Unfreezing base layers during fine tuning can cause catastrophic forgetting of pre-trained features. Hence it requires larger initial weight updates or smaller learning rate and more epochs to converge.

13. **Training and Validation Loss for Feature Extraction and Fine Tuning**

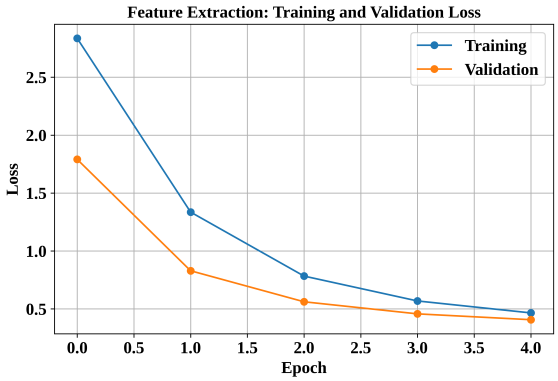


Figure 17: Feature Extraction

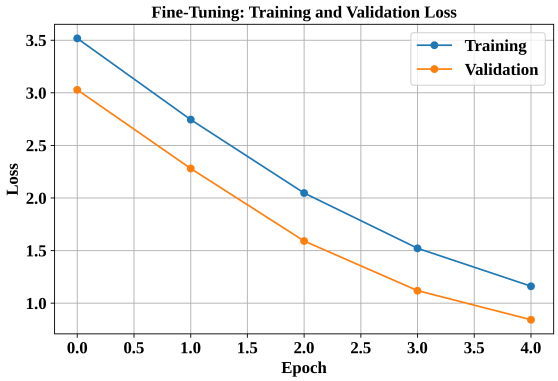


Figure 18: Fine Tuning

Feature Extraction: Training and Validation Loss vs Epoch Inference:
The plot shows the steady decrease in Training and Validaiton loss curves for feature extraction. The training loss is higher than validation loss in the earlier epochs. By the end of 4th epoch, both the loss values converge at 0.5. The steady decline indicates that the model is actively learning target features without overfitting. Validation loss is often lower than training loss because training loss is averaged during an epoch while the model is still updating, whereas validation loss is measured after the epoch's weight updates.

Fine Tuning: Training and Validation Loss vs Epoch Inference:
The plot shows the Training and Validaiton loss curves for fine tuning. Both training and validation loss decrease smoothly and consistently over time. Validation loss remains consistently lower than training loss throughout all epochs. The steady decline indicates that the model is actively learning target features without overfitting.

14. **5-Fold Cross Validation Accuracy**

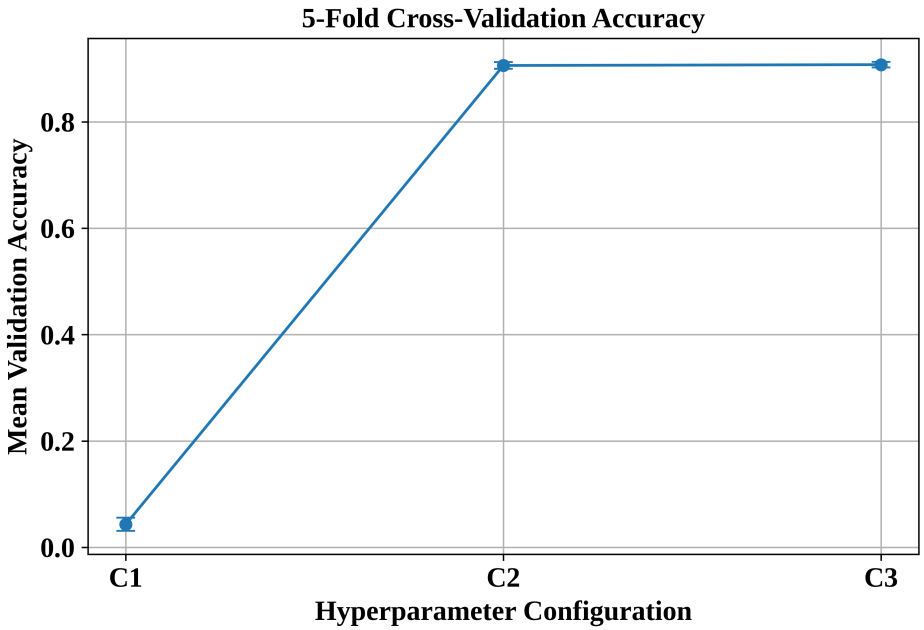


Figure 19: 5-Fold Cross Validation Accuracy

Inference:
The plot shows the mean validation accuracy for 3 different hyperparameter configurations. C1 has the lowest validation accuracy whereas C3 has the highest validation accuracy. The configuration are as follows:
C1 (Learning rate: 0.0001, Batch size: 16, Dropout:0.25, Optimizer: SGD)
C2 (Learning rate: 0.001, Batch size: 16, Dropout:0.25, Optimizer: Adam)
C3 (Learning rate: 0.001, Batch size: 32, Dropout:0.5, Optimizer: Adam)
C1 performs poorly as it uses SGD optimizer. Whereas Adam adaptively calculates individual learning rates for every parameter using momentum, allowing it to converge faster and require far less manual tuning than SGD.

15. **Confusion Matrix**

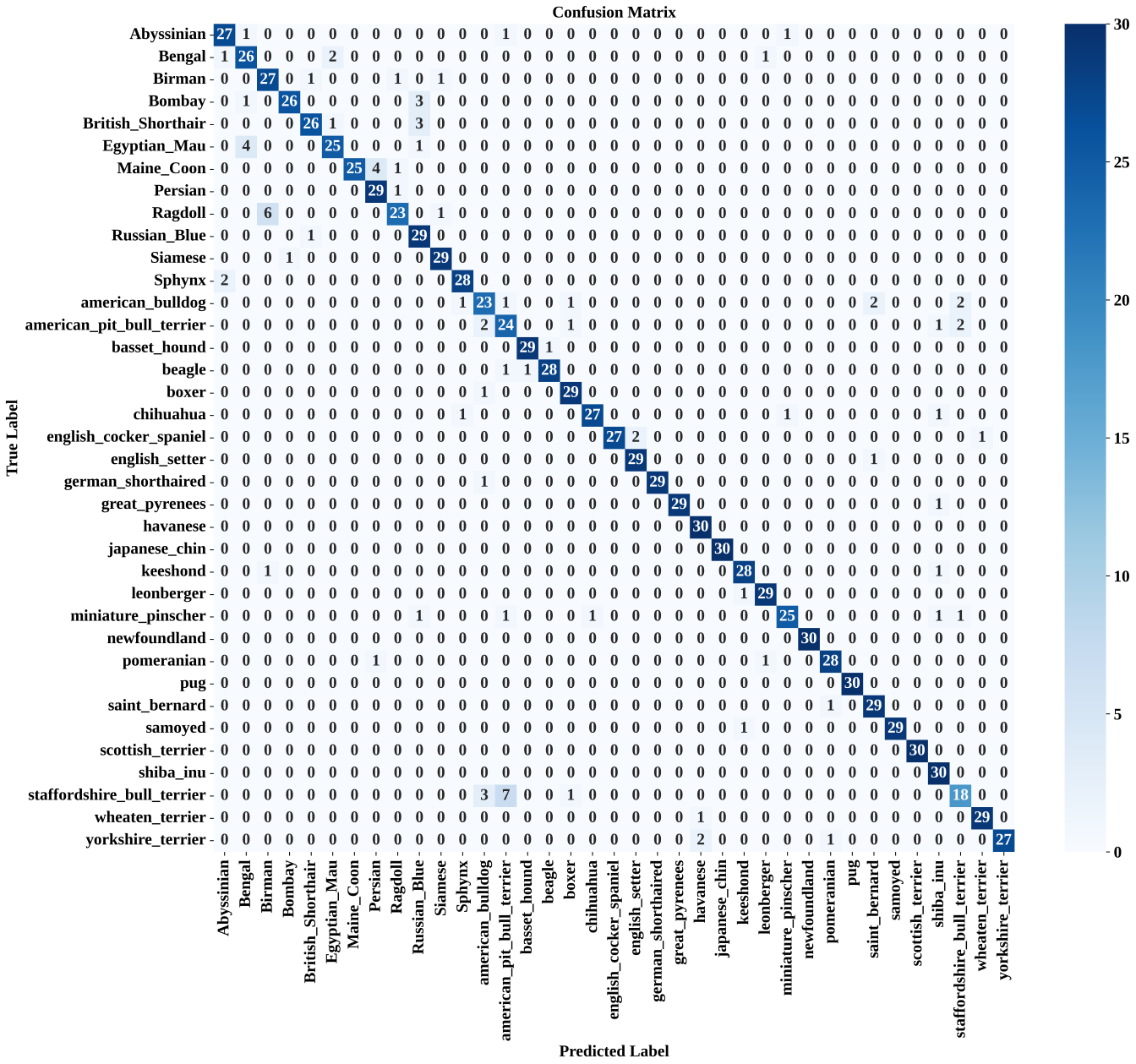


Figure 20: ConfusionMatrix

Inference:
The confusion matrix shows the performance of image classification model on 37 different pet breeds. Minor misclassifications are rare and occurs between visually similar pet breeds such as Staffordshire Bull Terriers are misclassified as Pit Bulls and Ragdolls with Birmans. This trend occurs because while the model easily learns distinct breed features, closely related breeds share nearly identical physical features.

16. Misclassified Images

Figure 21: Misclassified Images

Inference:
The figure shows some of the images that has been misclassified by the model. From the misclassified images, we can see that cat images are mapped to cat breeds, and dog images are mapped to dog breeds. However, there has been a class mismatch of cat breeds and dog breeds for cat and dog images respectively. It is also observed that a chihuahua has been misclassified as a sphynx which is a cat breed.

15 Results

Optimization Algorithms Report

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Training Time (in seconds)
SGD	3.498174	0.066727	10	219.736567
Momentum	1.418184	0.741208	10	203.661996
RMSProp	0.122968	0.912534	8	260.758930
Adam	0.149744	0.899008	9	208.824780

K-Fold Cross Validation Report

Configuration	Accuracy					Mean ± SD
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
C1	0.0338	0.0444	0.0290	0.0648	0.0445	0.0433 ± 0.0123
C2	0.906280	0.898551	0.915861	0.900387	0.909091	0.9060 ± 0.0062
C3	0.899517	0.907246	0.906190	0.909091	0.915861	0.9076 ± 0.0053

Final Model Evaluation Report

Metric	Value
Mean CV Accuracy	0.907581
CV Standard Deviation	0.005252
Test Accuracy	0.9161406755447388
Precision	0.919625783910535
Recall	0.9161406672678089
F1-score	0.9160590223088639
Training Time	251 seconds
Number of Parameters	2426725

Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	0.8918	0.0087	0.8952	164 seconds
Best Initialization	0.9085	0.0069	0.9038	127 seconds
Best Regularization	0.9333	0.0048	0.9287	137 seconds
Best Optimizer	0.9196	0.0061	0.9145	261 seconds
Best Hyperparameters	0.9254	0.0057	0.9218	250 seconds
Fine-Tuned Model	0.9076	0.0053	0.9161	251 seconds

16 Discussion Questions

1. What is the difference between model parameters and hyperparameters?
2. Why is weight initialization important?
3. Why can zero initialization be problematic for neural networks?
4. Compare Xavier and He initialization.
5. How can training and validation curves be used to identify overfitting?
6. How does Dropout reduce overfitting?
7. What is the purpose of Batch Normalization?
8. Explain the numerical Batch Normalization example.
9. What are the roles of γ and β?
10. Compare SGD, Momentum, RMSProp and Adam.
11. What happens when the learning rate is too large?
12. What happens when the learning rate is too small?
13. What is the effect of increasing batch size?
14. Explain stride and padding.
15. Why is MobileNetV2 computationally efficient?
16. What is depthwise separable convolution?
17. What is transfer learning?
18. Differentiate feature extraction and fine-tuning.
19. Why is a smaller learning rate generally used during fine-tuning?
20. Why is K-Fold Cross-Validation useful for hyperparameter selection?
21. Why must the test set remain untouched during tuning?
22. Why should mean and standard deviation both be reported?
23. Is the highest validation accuracy always sufficient to select a model?

17 Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

18 Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

19 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`